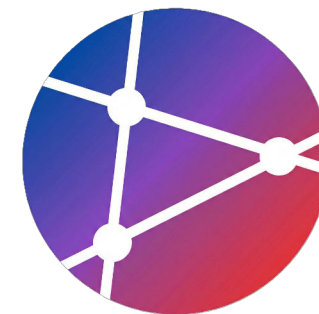




Teamwork & Collaborative Development

팀워크와 협업 개발

Pull Request 기반 워크플로우, 코드 리뷰 전략, 그리고 협업 베스트 프랙티스

MAIN SECTION

팀워크와 협업 개발

Teamwork and Collaborative Development



SECTION 02

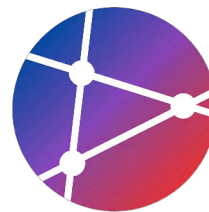
학습 목표

Learning Objectives

Week
3

AI Open
Source
Software





학습 목표

Learning Objectives for Week 3



협업 워크플로우 구축

Pull Request 기반의 체계적인 협업 프로세스를 이해하고 실무에 적용할 수 있습니다.



효과적인 코드 리뷰

팀의 생산성과 코드 품질을 높이는 건설적인 코드 리뷰를 수행할 수 있습니다.



Git 브랜치 전략

Git Flow와 GitHub Flow 등 다양한 브랜치 전략을 이해하고 상황에 맞게 선택할 수 있습니다.



협업 베스트 프랙티스

지속적인 통합과 배포, 문서화 등 원활한 팀 협업을 위한 모범 사례를 실천합니다.

SECTION 02

왜 협업이 중요한가?

Why Collaboration Matters?

Week
3

Teamwork
and
Collaborative
Development



혼자 vs 팀 (Solo vs Team)

Why collaboration matters: Comparing development approaches

혼자 개발 (Solo Dev)

- ✗ 제한된 관점 (Limited Perspective)
- ✗ 지식 사일로 (Knowledge Silo)
- ✗ 단일 실패 지점 (SPOF)
- ✗ 느린 피드백 (Slow Feedback)

⚠ 버스 팩터 (Bus Factor) = 1

팀 협업 (Team Collaboration)

- ✓ 다양한 관점과 아이디어
- ✓ 지식 공유 및 상호 학습
- ✓ 위험 분산 (Risk Distribution)
- ✓ 빠른 피드백 루프
- ✓ 더 나은 코드 품질 (Quality)

Bus Factor (버스 팩터)

“ 팀원 몇 명이 갑자기 프로젝트에서 이탈했을 때(버스에 치이는 등), 프로젝트가 중단되거나 망하게 되는가?

⚠ Danger Zone



$$BF = 1$$

👤 특정 1명에게 지식 집중

☠ 단일 실패 지점 (SPOF)

✈ 휴가/퇴사 시 프로젝트 마비

🚫 인수인계 불가능한 상태

🛡 Safe Zone



$$BF \geq 3$$

🔗 지식 공유 활성화

👥 페어 프로그래밍 문화

📖 철저한 문서화

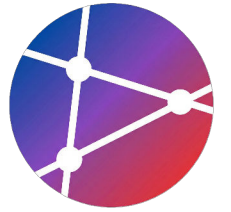
♻ 언제든지 대체 가능한 인력

MAIN SECTION

Git

협업 워크플로우

Git Collaboration Workflow



Git Flow vs GitHub Flow

Overview of Branching Strategies



Git Flow (Traditional)

복잡하지만 체계적인 구조

- 정기적인 릴리스 주기를 가진 대규모 팀에 적합
- 다양한 환경(Dev, Staging, Prod) 관리에 유리
- 명확한 역할 분담과 안전한 배포 프로세스



GitHub Flow (Modern) **RECOMMENDED**

단순하고 직관적인 구조

- 지속적 배포(CI/CD)에 최적화된 워크플로우
- 작은 팀, 스타트업, 빠른 개발 주기에 적합
- **Main** 브랜치는 항상 배포 가능한 상태 유지

Git Flow (전통적)

Robust branching strategy for scheduled releases

GIT FLOW STRUCTURE

```
main (프로덕션)
├── develop (개발)
│   ├── feature/login
│   ├── feature/signup
│   └── release/v1.0
│       └── hotfix/critical-bug
└── hotfix → main
```

주요 특징

- ✓ 복잡하지만 매우 체계적인 구조
- ✓ 대규모 팀 및 정기적인 릴리스 주기에 적합
- ✓ 명확한 환경 분리 지원 (Dev, Staging, Prod)
- ✓ 버전 관리 및 이력 추적이 용이함

브랜치 구조

main	언제나 배포 가능한 프로덕션 코드
develop	다음 버전을 위한 개발 통합 브랜치
feature/*	단위 기능 개발 (develop에서 분기)
release/*	배포 전 QA 및 준비 (develop에서 분기)
hotfix/*	운영 중 긴급 버그 수정 (main에서 분기)

GitHub Flow (현대적) ★ 권장

Modern collaboration workflow: Optimized for continuous deployment and simplicity

main (Production)

feature/user-auth

PR → Review → Merge → Deploy

fix/login-bug

PR → Review → Merge → Deploy

docs/update-readme

PR → Review → Merge → Deploy

⚡ 주요 특징

- ✓ 단순하고 직관적 구조
- ✓ 지속적 배포(CD) 최적화
- ✓ 작은 팀/스타트업 적합
- ✓ Main = Always Deployable

☰ 핵심 규칙

- 1 **main** 브랜치는 항상 배포 가능한 상태 유지
- 2 새 작업은 항상 **main**에서 브랜치 생성
- 3 브랜치는 **설명적인 이름** 사용 (feature/*)
- 4 **Pull Request**를 통해 코드 리뷰 및 논의
- 5 리뷰 승인 후 **Merge** 및 즉시 **Deploy**

브랜치 네이밍 컨벤션

Naming conventions for clear communication and organized history

Prefix Types

feature/ 새로운 기능 개발 (New features)

fix/ 버그 수정 (Bug fixes)

docs/ 문서 작업 (Documentation)

refactor/ 코드 리팩토링 (No logic change)

test/ 테스트 코드 추가/수정

chore/ 빌드, 설정 등 기타 작업

git-bash — examples

```
$ git checkout -b feature/user-authentication
$ git checkout -b fix/login-validation-error
$ git checkout -b docs/api-documentation
$ git checkout -b refactor/user-service
$ git checkout -b test/add-integration-tests
$ git checkout -b chore/update-dependencies
```

MAIN SECTION

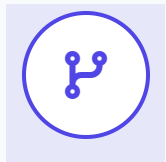
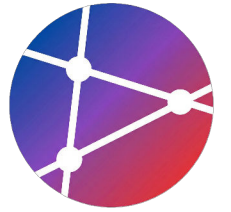
Pull Request 마스터하기

Mastering Pull Requests

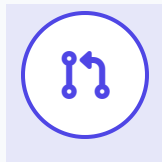
Week 3
AI Open Source Software

Pull Request란?

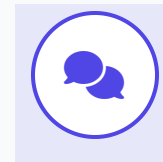
협업과 코드 품질을 위한 핵심 프로세스



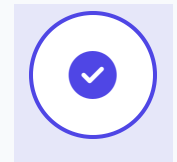
Your Branch



Pull Request



Code Review



Merge to Main



코드 리뷰 요청

작업한 변경 사항을 팀원들에게 알리고 검토를 요청합니다.



변경사항 논의

코드의 구현 방식, 설계, 개선점에 대해 토론합니다.



품질 검증

버그를 사전에 방지하고 일관된 코드 스타일을 유지합니다.



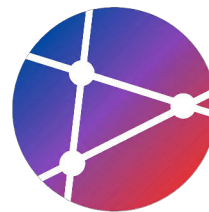
지식 공유

팀원 간의 기술적 노하우와 프로젝트 이해도를 공유합니다.



문서화

코드 변경의 이력과 맥락을 남겨 추후 유지보수를 돕습니다.



완벽한 PR 만들기

Key Elements of a High-Quality Pull Request

H

1. 좋은 PR 제목

명확하고 설명적인 제목은 리뷰어에게 변경 사항의 목적을 즉시 전달하며, 검색과 관리의 효율성을 높입니다.



2. 상세한 PR Description

변경 내용, 테스트 방법, 스크린샷, 관련 이슈 등을 상세히 기록하여 리뷰어에게 충분한 맥락을 제공합니다.



3. PR 템플릿 활용

.github/pull_request_template.md를 사용하여 팀 내 일관된 문서 양식을 유지하고 필수 항목 누락을 방지합니다.

좋은 PR 제목 (Good PR Titles)

Clear and descriptive titles significantly improve review efficiency

❌ 나쁜 예 (Bad Examples)

❌ "Update"

❌ "Fix bug"

❌ "Changes"

❗ Why bad?

너무 모호하여 리뷰어가 코드를 열어보기 전까지 내용을 짐작할 수 없습니다.

✅ 좋은 예 (Good Examples)

✅ [Feature] Add user authentication with JWT

✅ [Fix] Resolve login validation error on empty email

✅ [Refactor] Extract user service logic to separate module

✅ [Docs] Update API documentation for v2.0

2. 상세한 PR Description

Providing context and verification details for better reviews

📄 Context & Changes

📋 변경 사항

이 PR이 무엇을 하는지 간단히 설명

🎯 관련 Issue

- Closes #123
- Relates to #456

🔍 변경 내용

- 사용자 인증 API 엔드포인트 추가
- JWT 토큰 생성 및 검증 로직 구현
- 비밀번호 해싱 (bcrypt 사용)
- 인증 미들웨어 추가

📝 추가 노트

기타 리뷰어가 알아야 할 정보

☑️ Verification & Checklist

✏️ 테스트

- ☐ Unit 테스트 추가
- ☐ Integration 테스트 추가
- ☒ 로컬에서 테스트 완료
- ☒ CI 파이프라인 통과

📸 스크린샷

[스크린샷 첨부 (UI 변경 시 필수)]

✅ 체크리스트

- ☒ 코드가 스타일 가이드를 따름
- ☒ 자체 리뷰 완료
- ☐ 주석 추가 (복잡한 로직)
- ☐ 문서 업데이트
- ☒ Breaking changes 없음

PR 템플릿 생성

Standardizing Pull Requests with Templates



파일 위치

.github/pull_request_template.md



핵심 구성 요소

- 변경 사항 요약 (Summary)
- 변경 타입 (Type)
- 관련 Issue (Link)
- 테스트 계획 (Test Plan)
- 체크리스트 (Checklist)



템플릿 효과

PR 생성 시 자동으로 내용이 채워져 작성 시간을 단축하고 팀 내 일관성을 유지합니다.

pull_request_template.md

변경 사항 요약

<!-- 무엇을 변경했는지 간단히 설명 -->

변경 타입

- | | |
|---------------------|-----------------------|
| - [] 🐛 Bug fix | - [] 📄 Documentation |
| - [] ✨ New feature | - [] 🎨 Style |
| - [] 🔧 Refactoring | - [] ✅ Test |
| - [] 🛠 Chore | |

관련 Issue

<!-- Closes #이슈번호 -->

상세 설명

<!-- 구체적인 변경 내용 -->

테스트 계획

<!-- 어떻게 테스트했는지 -->

스크린샷

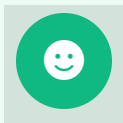
<!-- UI 변경이 있다면 -->

체크리스트

- | | |
|-----------------|---------------|
| - [] 코드 리뷰 완료 | - [] 문서 업데이트 |
| - [] 테스트 추가/수정 | - [] CI 통과 |

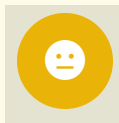
PR 크기 가이드

Pull Request Size & Review Efficiency



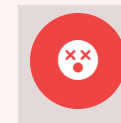
Small PR

- 🕒 빠른 리뷰 완료 (< 1시간)
- 🛡️ 높은 코드 품질 유지
- 👍 적극 권장!



Medium PR

- 🕒 리뷰 시간 다소 증가
- ✅ 여전히 관리 가능한 수준
- ! 복잡도에 따라 주의 필요



Large PR

- ⌚ 리뷰하기 매우 어려움
- 🚨 오류 발생 가능성 높음
- ✂️ 여러 PR로 분할 필수



Golden Rule

"작은 PR을 자주 올리세요 (Small Batches, Frequent Updates)"

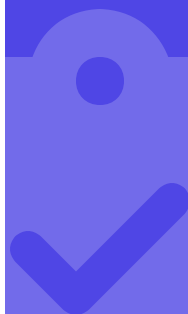
Ship Small Diffs

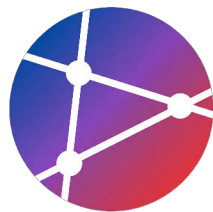
MAIN SECTION

효과적인 코드 리뷰

Effective Code Review Practices

Week 3
AI Open Source Software





코드 리뷰의 목적

Why Code Reviews Matter



버그 발견

잠재적인 오류와 엣지 케이스를 사전에 식별하여 시스템 안정성을 확보합니다.



코드 품질 향상

가독성, 유지보수성, 성능 최적화를 통해 더 견고한 코드베이스를 구축합니다.



지식 공유

코드 패턴과 도메인 지식을 공유하여 특정인에게 지식이 편중되는 것을 방지합니다.



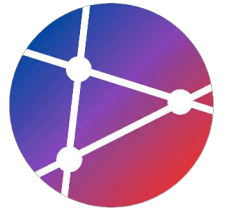
일관성 유지

코딩 컨벤션과 아키텍처 스타일을 통일하여 협업 효율성을 높입니다.



팀 문화 형성

상호 피드백을 통해 함께 성장하고 소통하는 건강한 개발 문화를 만듭니다.



리뷰어로서 해야 할 것

Responsibilities of an Effective Code Reviewer



1. 체계적으로 리뷰하기

체크리스트를 기반으로 코드 품질, 테스트 커버리지, 보안 취약점 등을 꼼꼼하게 검증합니다.



2. 건설적인 피드백

비난이 아닌 개선을 위한 구체적인 제안과 대안을 제시하여 동료의 성장을 돕습니다.



3. 코멘트 태그 사용

[MUST], [SHOULD], [NITS] 등의 태그를 사용하여 피드백의 중요도와 의도를 명확히 전달합니다.

체계적으로 리뷰하기 (Systematic Review)

Reviewer's Checklist: Ensuring Code Quality and Stability

⚙️ 기능 및 신뢰성 (Functionality)



코드가 요구사항을 충족하는가?

Does the code meet all functional requirements?



테스트가 충분한가?

Is there sufficient unit and integration test coverage?



버그나 오류가 없는가?

Are there any obvious logic errors or edge cases missed?



성능 문제는 없는가?

Are there any N+1 queries or inefficient algorithms?

🛡️ 품질 및 유지보수 (Quality)



보안 취약점은 없는가?

Check for SQL injection, XSS, and data exposure.



코드가 읽기 쉬운가?

Is the code clean, readable, and self-explanatory?



일관된 스타일을 따르는가?

Does it follow the team's coding conventions and linting rules?



문서화가 충분한가?

Are complex logic and public APIs well-documented?

건설적인 피드백 (Constructive Feedback)

How to provide effective code review comments

👎 나쁜 코멘트 (Bad Comments)

❌ "이 코드는 끔찍해요."

비난적이고 구체적이지 않음

❌ "왜 이렇게 했나요?"

공격적으로 들릴 수 있는 질문

❌ "다시 작성하세요."

이유나 방향 제시 없음

👍 좋은 코멘트 (Good Comments)

✅ "이 부분을 함수로 추출하면 재사용성이 높아질 것 같습니다."

구체적인 제안과 이유 설명

✅ "성능을 위해 이 루프를 최적화하는 건 어떨까요?"

부드러운 제안형 어조 (Open Question)

✅ "이 로직에 대한 주석을 추가하면 이해하기 더 쉬울 것 같습니다."

개선 효과 언급

3. 코멘트 태그 사용 (Comment Tags)

Clarifying intent and priority in code review feedback

Tag Definitions

[MUST]

반드시 수정이 필요한 사항. 로직 오류, 보안 취약점, 스타일 가이드 위반 등.

[SHOULD]

수정을 강력히 권장하는 사항. 더 나은 방법이나 개선안이 있을 때.

[NITS]

사소한 제안 ("Nitpick"). 공백, 오타, 개인적 선호 등. 수정하지 않아도 무방함.

[IDEA]

미래를 위한 아이디어 제안. 이번 PR의 범위를 벗어날 수 있음.

[QUESTION]

코드 동작이나 의도에 대한 질문. 명확하지 않은 부분 확인.

[PRAISE]

잘 작성된 코드에 대한 칭찬과 격려. 긍정적 피드백.

Usage Examples

[MUST]

이 함수는 `null` 입력에 대한 체크가 빠져있어 런타임 에러가 발생할 수 있습니다.

[SHOULD]

변수명 `val`은 의미가 모호합니다. `userInputValue`로 변경하면 더 명확할 것 같습니다.

[NITS]

여기 불필요한 공백 라인이 2줄 있네요. 제거 부탁드립니다.

[IDEA]

이 유효성 검사 로직을 별도 유틸리티 함수로 분리하면 다른 컴포넌트에서도 쓸 수 있을 것 같아요.

[QUESTION]

이 루프가 데이터가 100만 건일 때도 성능 이슈 없이 동작할까요? 엡지 케이스 처리가 궁금합니다.

[PRAISE]

테스트 커버리지가 완벽하네요! 예외 케이스까지 꼼꼼하게 처리한

리뷰 준비 (Preparing for Review)

PR 제출 전 반드시 확인해야 할 체크리스트



**Before You
Submit PR**

- ✓ **자체 리뷰 먼저 (Self-Review)**
코드 제출 전 스스로 먼저 꼼꼼히 검토하기
- ✓ **테스트 실행 및 통과 확인**
모든 Unit/Integration 테스트가 통과하는지 확인
- ✓ **Lint / Formatter 통과**
코드 스타일 가이드 준수 여부 자동 검사
- ✓ **명확한 설명 작성 (Description)**
변경 사항의 목적과 내용을 명확히 기술
- ✓ **작은 단위로 분할 (Small Batches)**
리뷰어가 소화하기 쉬운 크기로 작업 쪼개기

피드백 수용 태도 (Attitude)

Best practices for receiving code review feedback constructively

👍 좋은 태도 (Do's)

- ✓ 감사 표시 (Say Thank You)
- ✓ 질문에 답변 (Answer Questions)
- ✓ 건설적으로 토론 (Discussion)
- ✓ 신속한 수정 (Quick Fixes)
- ✓ 배우는 자세 (Open Mind)

❗ 피해야 할 태도 (Don'ts)

- ✗ 방어적 태도 (Defensive)
- ✗ 피드백 무시 (Ignore)
- ✗ 감정적 반응 (Emotional)
- ✗ 반영 지연 (Delay)

코드 리뷰 예시 (Example)

[</> Original Code](#)

Code Under Review: Calculation Logic

```
cart.js

// 리뷰 대상 코드: 총액 계산 함수
function calculateTotal(items) {
  let total = 0;
  for (let i = 0; i < items.length; i++) {
    total += items[i].price * items[i].quantity;
  }
  return total;
}
```

코드 리뷰 예시 (Review Example)

Constructive feedback using clear tags and actionable suggestions



@reviewer commented just now

[QUESTION]

`items` 가 빈 배열이거나 `null` 인 경우 처리가 필요한가요?

[NITS]

함수 설명 주석(Docstring)을 추가하면 이해하기 더 쉬울 것 같습니다.

[SHOULD]

현대적인 배열 메서드(`reduce`)를 사용하면 코드가 더 선언적이고 읽기 쉬워질 것 같습니다.

→ [우측의 개선된 코드 참조](#)

Js suggested_refactor.js

JavaScript

```
// Refactored implementation
function calculateTotal(items) {
  return items.reduce((total, item) =>
    total + (item.price * item.quantity),
    0
  );
}
```

MAIN SECTION

자동화된 코드 리뷰

Automated Code Review & CI/CD



GitHub Actions로 자동 체크

Automated Code Review Workflow

✓ Lint Code

ESLint를 실행하여 코드 스타일과 문법 오류를 자동으로 검사합니다. 실패 시 PR에 코멘트를 남깁니다.

🔧 Run Tests

단위 테스트(npm test)를 실행하고 Codecov를 통해 커버리지 리포트를 생성합니다.

🛡 Security Scan

Snyk 등의 도구를 사용하여 의존성 패키지의 보안 취약점을 스캔합니다.

📊 Check PR Size

변경 라인 수에 따라 XS, S, M, L 등의 라벨을 자동으로 부착하여 리뷰 난이도를 표시합니다.

.github/workflows/code-review.yml

```
name: Automated Code Review on: pull_request: types: [opened, synchronize] jobs: lint:
  name: Lint Code runs-on: ubuntu-latest steps: - uses: actions/checkout@v3 - name: Run
  ESLint run: npm run lint - name: Comment on PR if: failure() uses: actions/github-
  script@v6 with: script: | github.rest.issues.createComment({ issue_number:
  context.issue.number, owner: context.repo.owner, repo: context.repo.repo, body: '❌
  Linting failed. Please fix issues.' }) test: name: Run Tests runs-on: ubuntu-latest
  steps: - uses: actions/checkout@v3 - name: Run Tests run: npm test security: name:
  Security Scan runs-on: ubuntu-latest steps: - uses: actions/checkout@v3 - name: Run
  Security Scan uses: snyk/actions/node@master env: SNYK_TOKEN: ${ secrets.SNYK_TOKEN
  }}
```

Code Owners 설정

Defining ownership for automated review requests (.github/CODEOWNERS)

CODEOWNERS

```
# .github/CODEOWNERS 파일 설정 예시
# 기본: 모든 파일에 대한 소유권 (Fallback)
* @team-lead
# 프론트엔드 팀 소유권
/src/components/** @frontend-team
/src/pages/** @frontend-team
# 백엔드 팀 소유권
/src/api/** @backend-team
/src/services/** @backend-team
# 데이터베이스 관련 (복수 팀 할당 가능)
/migrations/** @database-admin @backend-team
# 인프라 및 CI/CD 설정
/infrastructure/** @devops-team
/.github/workflows/** @devops-team
# 문서화
```


자동 리뷰 요청 (Auto Review Request)

Assigning reviewers automatically based on configuration

```
.github/auto_assign.yml

1 # .github/auto_assign.yml
2 # PR 생성 시 자동으로 리뷰어 할당
3
4 reviewers:
5 - octocat
6 - hubot
7 - other-user
8
9 numberOfReviewers: 2
10
11 # 파일 패턴별 리뷰어 그룹 설정
12 reviewGroups:
13   frontend:
14     - frontend-dev1
15     - frontend-dev2
16   backend:
```

SECTION 07

커밋 메시지 컨벤션

Conventional Commits & Best Practices

Conventional Commits

A specification for adding human and machine readable meaning to commit messages



Type (타입) 분류

Standard conventional commit types for clear history

 feat 새로운 기능 추가 (New feature)	 fix 버그 수정 (Bug fix)	 refactor 코드 리팩토링 (기능/로직 변경 없음)
 perf 성능 개선 (Performance improvements)	 test 테스트 추가/수정 (Adding missing tests)	 style 코드 포매팅 (세미콜론, 공백 등)
 docs 문서 변경 (README, Wiki 등)	 chore 빌드/설정 변경 (패키지 매니저 설정 등)	 ci CI 설정 변경 (Actions, CircleCI 등)

좋은 커밋 메시지 예시

Examples of well-structured Git commit messages

상세 (Detailed)

RECOMMENDED

```
feat(auth): implement JWT-based authentication
Implement JSON Web Token authentication for API endpoints.
- Add login endpoint
- Add token validation middleware
- Update user model with password hashing
Closes #123
```

기본 (Basic)

```
feat: add user authentication
```

⚠ Breaking Change

IMPORTANT

```
feat(api)!: change user endpoint response format
BREAKING CHANGE: User API now returns data in
camelCase instead of snake_case
```

커밋 메시지 자동 검증 워크플로우

Automating Conventional Commits validation with GitHub Actions

  .github/workflows/commit-lint.yml

```
1 name: Commit Lint
2
3 on:
4   pull_request:
5     types: [opened, synchronize]
6
7 jobs:
8   lint-commits:
9     runs-on: ubuntu-latest
10    steps:
11      - uses: actions/checkout@v3
12        with:
13          fetch-depth: 0 # 모든 히스토리를 가져와야 커밋 린트가 가능함
14
15      - name: Lint commit messages
16        uses: wagoid/commitlint-github-action@v5
```

커밋 메시지 자동 검증 (2/2)

commitlint.config.js 설정 파일을 통한 규칙 정의

```
commitlint.config.js

// commitlint.config.js
module.exports = {
  extends: ['@commitlint/config-conventional'],
  rules: {
    'type-enum': [
      2,
      'always',
      ['feat', 'fix', 'docs', 'style', 'refactor', 'test', 'chore']
    ],
    'subject-case': [2, 'never', ['upper-case']],
    'subject-empty': [2, 'never'],
    'type-empty': [2, 'never']
  }
};
```

SECTION 08

Protected Branches

브랜치 보호 전략 및 보안 설정

Week
3

AI Open
Source
Software



Main 브랜치 보호 설정

Protected Branches: Ensuring code quality and security

Settings > Branches > Add branch protection rule

Branch name pattern

main

✓ Require pull request reviews before merging

- Required approving reviews: 2
- Dismiss stale approvals on new commits
- Require review from **Code Owners**

✓ Require conversation resolution

All conversations on code must be resolved before merging.

✓ Include administrators

Enforce all configured restrictions for administrators.

✓ Require status checks to pass before merging

- Require branches to be up to date
- Status checks: CI Tests Lint

✓ Require signed commits

Commits must have verified signatures.

✓ Restrict who can push to matching branches

Specify people, teams, or apps allowed to push.

Status Checks 예시

CI/CD Workflow for Branch Protection: Required Checks

 .github/workflows/required-checks.yml

YAML

```
name: Required Checks

on:
  pull_request:
  push:
    branches: [main]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Build
        run: npm run build

  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Test
        run: npm test

  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Lint
        run: npm run lint
```

SECTION 09

협업

베스트 프랙티스

Collaboration Best Practices



1. 소통 (Communication)

Collaboration Best Practices: Effective Communication

⊘ DON'T (지양할 태도)

- ⊘ 공격적이거나 방어적인 태도
- ⊘ 모호한 표현 (Vague Expressions)
- ⊘ 답변 지연 및 무시
- ⊘ 감정적인 반응 (Emotional Reactivity)

✔ DO (권장하는 태도)

- ✔ 명확하고 정중하게 표현하기
- 😊 이모지로 감정 전달 😊 (Soften Tone)
- ? 질문을 환영하는 분위기 조성
- 🕒 비동기 소통(Async) 고려
- ♥ 서로 존중하고 감사 표시하기

2. 문서화 (Documentation)

Collaborative Development Best Practices

Essential Documentation Checklist

✓ 아키텍처 결정

ADR

✓ API 명세

Specification

✓ 설정 방법

Setup Guide

✓ 트러블슈팅 가이드

Troubleshooting

✓ FAQ

Q&A

+ Contribution Guide

Optional

 **Tip:** 문서는 코드를 설명하는 것이 아니라, '왜'와 '어떻게'에 집중해야 합니다.

3. 페어 프로그래밍 (Pair Programming)

Collaborative coding: Two developers, one machine



Driver

(코딩하는 사람)

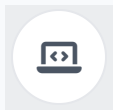
-  키보드를 잡고 **실제 코드를 작성**
-  현재의 문제 해결에 집중 (**전술적 사고**)
-  구문(Syntax) 및 세부 구현 담당



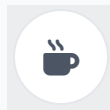
Navigator

(관찰하는 사람)

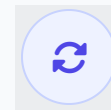
-  전체적인 **방향 제시 및 리뷰**
-  큰 그림을 보며 설계 검토 (**전략적 사고**)
-  실시간 오류 발견 및 대안 제시



CODING SESSION
25분 작업



SHORT BREAK
5분 휴식



SWITCH ROLES
역할 교대

4. 지식 공유 (Knowledge Sharing)

Building a culture of continuous learning and shared understanding

Interactive & Synchronous



Tech Talk (기술 발표)

정기적인 기술 세미나 및 노하우 공유



Lunch & Learn

점심 시간을 활용한 캐주얼한 학습 세션



Pair Programming

실시간 코드 작성과 즉각적인 피드백

Documentation & Review



문서화 (Documentation)

ADR, API 명세, 온보딩 가이드 작성



코드 리뷰 (Code Review)

코드 품질 향상 및 도메인 지식 전파



Internal Wiki

팀의 지식 베이스 구축 및 유지보수

MAIN SECTION

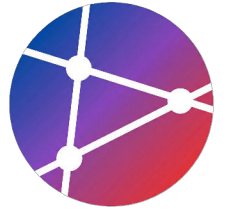
실습 과제 (Assignments)

Practical Exercises and Hands-on Labs



실습 과제: GitHub Flow

Hands-on Lab 01: Implementing Collaboration Workflow



Lab Task 01

Estimated Time: 20 min

i 이 실습의 목표는 Feature 브랜치 전략을 사용하여 안전하게 코드를 변경하고 PR을 통해 병합하는 전체 과정을 경험하는 것입니다.

1

Feature 브랜치 생성

메인 브랜치에서 새로운 작업 브랜치를 생성합니다.

```
git checkout -b feature/login-page
```

2

코드 변경 및 커밋

의미 있는 단위로 작업을 수행하고 Conventional Commits 규칙을 따릅니다.

```
git commit -m "feat: add login form UI"
```

3

Pull Request 생성

GitHub 저장소로 푸시 후 웹 인터페이스에서 PR을 생성합니다. 템플릿에 맞춰 설명을 작성하세요.

4

자체 리뷰 및 체크리스트

리뷰어를 지정하기 전에 스스로 코드를 검토하고 PR 체크리스트를 모두 완료합니다.

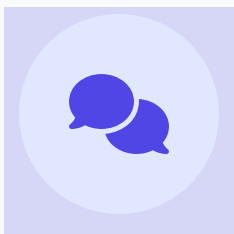
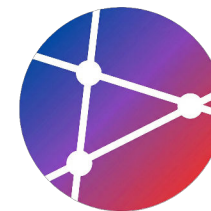
5

CI 통과 확인

자동화된 테스트와 Lint 검사가 통과되었는지 확인합니다 (All checks passed).

실습 과제: 코드 리뷰

Hands-on Lab 02: Constructive Code Review Practice



Lab Task 02

Estimated Time: 30 min

i 동료의 코드를 리뷰하며 품질을 개선하는 과정을 실습합니다. 비판이 아닌 개선을 위한 건설적인 피드백 문화를 경험해 보세요.

1

최소 3개의 PR 리뷰 🔍

현재 오픈되어 있는 다른 팀원의 Pull Request 중 최소 3개를 선택하여 리뷰를 시작합니다.

2

건설적인 피드백 제공

"왜?"라고 묻기보다는 대안을 제시하거나 의도를 파악하는 질문을 던집니다. 문제 해결 중심의 대화를 유도하세요.

3

리뷰 태그 시스템 활용 📌

중요도를 명확히 하기 위해 태그를 사용합니다.

[MUST]

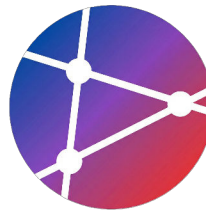
[SHOULD]

[NITS]

4

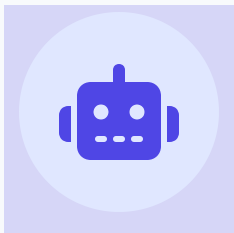
칭찬도 포함하기 (Praise) 👍

버그만 찾는 것이 아닙니다. 잘 작성된 코드나 기발한 아이디어에는 칭찬을 아끼지 마세요. [PRAISE]



실습 과제: 자동화 설정

Hands-on Lab 03: Setting up Automation & Governance



Lab Task 03

Estimated Time: 25 min

i GitHub의 자동화 기능과 정책 설정을 통해 코드 품질을 보장하고 협업 프로세스를 효율화하는 환경을 구축합니다.

1

PR 템플릿 생성

프로젝트 루트에 템플릿 파일을 생성하여 팀의 리뷰 표준을 정립합니다.

```
.github/pull_request_template.md
```

2

CODEOWNERS 설정

특정 디렉토리나 파일의 책임자를 지정하여 리뷰 요청을 자동화합니다.

```
* @your-github-id
```

3

Automated Checks 작성

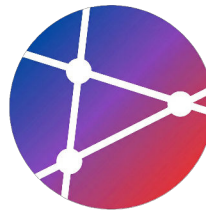
GitHub Actions를 사용하여 CI 워크플로우(Lint, Test)를 설정합니다.

```
.github/workflows/ci.yml
```

4

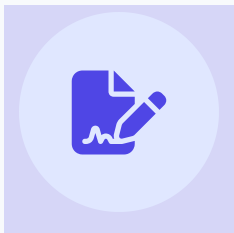
Branch Protection 설정

Main 브랜치에 직접 푸시를 막고, PR 리뷰와 Status Check 통과를 강제합니다.



실습 과제: 협업 문서 작성

Hands-on Lab 04: Documentation for Collaboration



Lab Task 04

Estimated Time: 30 min

i 이 실습의 목표는 팀의 원활한 협업을 위해 필요한 규칙과 가이드라인을 명시적으로 문서화하여 저장소에 공유하는 것입니다.

1

CONTRIBUTING.md 작성

프로젝트 루트에 기여 가이드 파일을 생성하고 개발 환경 설정 및 이슈 템플릿 정보를 작성합니다.

```
touch CONTRIBUTING.md
```

2

브랜치 전략 문서화

팀이 사용할 브랜치 전략(Git Flow 등)과 브랜치 명명 규칙을 도식화하여 정리합니다.

```
feature/*, fix/*, hotfix/*
```

3

코드 리뷰 가이드라인

리뷰어가 중점적으로 봐야 할 체크리스트와 [MUST], [SHOULD] 등 태그 사용법을 정의합니다.

4

커밋 메시지 컨벤션 문서

Conventional Commits 등 팀이 따를 커밋 메시지 형식을 예시와 함께 명시합니다.

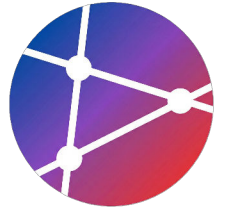
```
feat: add new login api
```

REFERENCES

참고 자료 및 추가 정보

Recommended Readings, Tools, and Resources





필수 읽기 (Essential Reading)

Recommended articles and guides for collaborative development



GitHub Flow Guide

WORKFLOW

GitHub에서 제안하는 경량화된 브랜치 기반 워크플로우에 대한 공식 가이드 문서입니다.

[Read Article](#)



Conventional Commits

CONVENTION

사람과 기계 모두가 읽기 쉬운 커밋 메시지를 작성하기 위한 명세와 규칙입니다.

[View Spec](#)



How to Write a Git Commit Message

BEST PRACTICE

좋은 커밋 메시지를 작성해야 하는 이유와 7가지 핵심 규칙을 설명한 Chris Beams의 글입니다.

[Read Post](#)

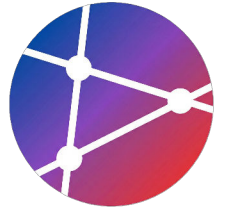


Code Review Best Practices

CODE REVIEW

Google 엔지니어링 팀에서 실천하는 효과적인 코드 리뷰 방법과 문화에 대한 가이드입니다.

[Read Guide](#)



협업 도구 (Collaboration Tools)

Recommended software and platforms for effective teamwork



PR 관리 (PR Management)

VERSION CONTROL

소스 코드 저장소 호스팅 및 Pull Request 기반의 협업을 지원하는 대표적인 플랫폼입니다.

 GitHub

 GitLab

 Bitbucket



코드 리뷰 (Code Review)

QUALITY ASSURANCE

체계적이고 심도 있는 코드 리뷰와 변경 사항 추적을 위한 전문 도구들입니다.

 ReviewNB

 Gerrit

 Phabricator



페어 프로그래밍 (Pair Programming)

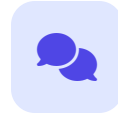
REAL-TIME COLLAB

원격 환경에서도 실시간으로 코드를 공유하고 함께 작성할 수 있는 도구입니다.

 VS Code Live Share

 Tuple

 Pop



커뮤니케이션 (Communication)

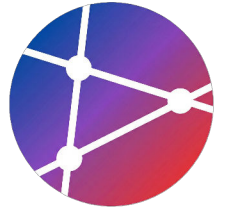
COMMUNICATION

팀 간의 빠르고 명확한 비동기/동기 소통을 지원하는 메신저 및 협업 툴입니다.

 Slack

 Discord

 Mattermost



추가 자료 (Additional Resources)

Recommended books for improving code quality and collaboration skills



READABILITY

The Art of Readable Code

Dustin Boswell

코드를 이해하기 쉽게 작성하는 구체적인 원리와 테크닉을 다룬 실무 지침서입니다. 변수 명명부터 제어 흐름 단순화까지 실용적인 팁을 제공합니다.



CONSTRUCTION

Code Complete

Steve McConnell

소프트웨어 구현에 관한 방대한 지식을 집대성한 필독서로, 설계, 코딩, 디버깅, 테스트 등 개발의 전 과정을 체계적으로 다룹니다.



CLEAN CODE

Clean Code

Robert C. Martin

애자일 소프트웨어 장인 정신을 바탕으로, 깨끗하고 유지보수 가능한 코드를 작성하는 방법과 원칙을 제시하는 고전입니다.

CONCLUSION

핵심 요약

Key Takeaways

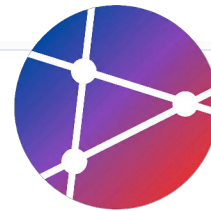
Summary & Future Directions



★ SUMMARY

Key Takeaways

성공적인 협업을 위해 이번 주차에서 배운 핵심 원칙들을 되짚어 봅니다.



01



작은 PR을 자주 (Small Batch Size)

빠른 리뷰와 피드백을 위해 변경 사항을 작게 유지하세요.

02



건설적인 코드 리뷰

비판이 아닌 배움과 품질 향상의 기회로 삼으세요.

03



명확한 커밋 메시지

동료와 미래의 자신을 위해 변경의 '이유'를 기록하세요.

04



자동화로 품질 보장 (CI/CD)

반복적인 검사는 기계에게 맡기고 로직에 집중하세요.

05

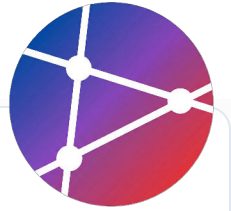


소통과 문서화

지식 공유는 팀의 버스 팩터를 높이는 가장 좋은 방법입니다.

Golden Rules

효율적이고 건강한 협업 문화를 만들기 위해 반드시 지켜야 할 5가지 핵심 원칙입니다.



01



PR은 200줄 이내로

리뷰어가 집중할 수 있는 Small Batch Size를 유지하세요.

02



리뷰는 24시간 이내에

동료의 작업이 블로킹되지 않도록 신속하게 피드백하세요.

03



커밋은 의미 단위로

하나의 커밋은 하나의 논리적 변경 사항만 포함해야 합니다.

04



브랜치는 설명적으로

feat/auth처럼 이름만으로 목적을 알 수 있게 작성하세요.

05



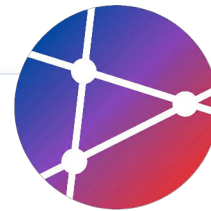
소통은 명확하고 정중하게

텍스트 기반 소통에서는 오해를 줄이기 위해 더욱 배려하세요.

WRAP UP

Next Steps

오늘 배운 협업 원칙을 실무에 적용하기 위한 구체적인 실행 계획입니다.



01



GitHub Flow 적용

팀 프로젝트에 브랜치 전략을 도입하고 실천합니다.

02



코드 리뷰 문화 형성

상호 존중 기반의 리뷰로 코드 품질을 높입니다.

03



자동화 파이프라인 구축

CI/CD를 통해 반복 작업을 줄이고 안정성을 확보합니다.

04



팀 컨벤션 정립

일관된 코딩 스타일과 커밋 메시지 규칙을 만듭니다.

66

"Code is read more often than it is written."

- Guido van Rossum